



Instituto Tecnológico de Mazatlán



**Sensor ambiental de precisión BME280/BMP280:
estación barométrica con Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarria González

1. Introducción

El BMP280 es un sensor digital de presión barométrica y temperatura fabricado por Bosch Sensortec. Mide la presión atmosférica en un rango de 300 a 1100 hPa y la temperatura de -40 a 85°C , y entrega los datos por I2C o SPI. El BME280 es su versión ampliada: usa el mismo encapsulado, la misma distribución de registros y las mismas direcciones, pero además mide humedad relativa. Para distinguirlos, cada chip guarda un identificador en el registro 0xD0: 0x58 en el BMP280 y 0x60 en el BME280.

Como la presión atmosférica disminuye al subir, a partir de ella se puede estimar la altitud. La librería de Adafruit usa la fórmula barométrica internacional, $h = 44330 \cdot [1 - (P / P_0)^{0.1903}]$, donde P es la presión medida y P_0 la presión a nivel del mar del día. Por eso la altitud calculada depende directamente del valor de P_0 que se configure en el programa.

En esta práctica se conectó un módulo BMP280 a un Arduino UNO R4 WiFi por el bus I2C para construir una pequeña estación barométrica: el programa lee temperatura, presión y altitud cada segundo, envía los datos al Monitor Serie en un formato compatible con el Serial Plotter y muestra en la matriz LED de 12×8 de la placa la temperatura y la altitud de forma alternada. Toda la temporización se hace con `millis()`, sin usar `delay()`.

2. Objetivos

Objetivo general:

- Leer temperatura, presión atmosférica y altitud con el sensor BMP280/BME280 por I2C y mostrarlas en el Monitor Serie y en la matriz LED del Arduino UNO R4 WiFi.

Objetivos específicos:

- Conectar el sensor al bus I2C del Arduino (SDA y SCL) y alimentarlo con 3.3 V.
- Detectar el sensor en las direcciones 0x76 y 0x77 e identificarlo leyendo su Chip ID en el registro 0xD0.
- Configurar el muestreo del sensor (modo normal, sobremuestreo y filtro IIR) con la librería Adafruit BMP280.
- Calcular la altitud a partir de la presión y de la presión a nivel del mar del día.
- Mostrar los datos en el Monitor Serie y en el Serial Plotter, y alternar temperatura y altitud en la matriz LED.
- Manejar tres tareas con `millis()` (lectura, matriz y reconexión) y detectar lecturas inválidas cuando el sensor se desconecta.

3. Herramientas y material utilizado

- 1 Arduino UNO R4 WiFi y su cable USB-C
- 1 módulo sensor BMP280/BME280 de 6 pines (VCC, GND, SCL, SDA, CSB y SDO)
- 4 cables de conexión (jumpers)
- Arduino IDE 2 con las librerías Adafruit BMP280 Library, Adafruit Unified Sensor y ArduinoGraphics; Arduino_LED_Matrix ya viene incluida con la placa R4
- Monitor Serie y Serial Plotter a 115200 baudios

4. Desarrollo / Metodología

Primero se instalaron desde el Gestor de librerías la **Adafruit BMP280 Library**, la **Adafruit Unified Sensor** y la **ArduinoGraphics**. Después se conectó el sensor al Arduino con cuatro cables: VCC al pin de **3.3 V**, GND a tierra, y las líneas SDA y SCL a los pines SDA y SCL que están junto a AREF. En el UNO R4 WiFi esos pines

son los mismos que A4 (SDA) y A5 (SCL) y corresponden al bus wire; el conector Qwiic de la placa usa otro bus, wire1. Los pines CSB y SDO del módulo se dejaron sin conectar, por lo que el sensor trabaja en modo I2C y su dirección queda definida por la resistencia del propio módulo (0x76 o 0x77).

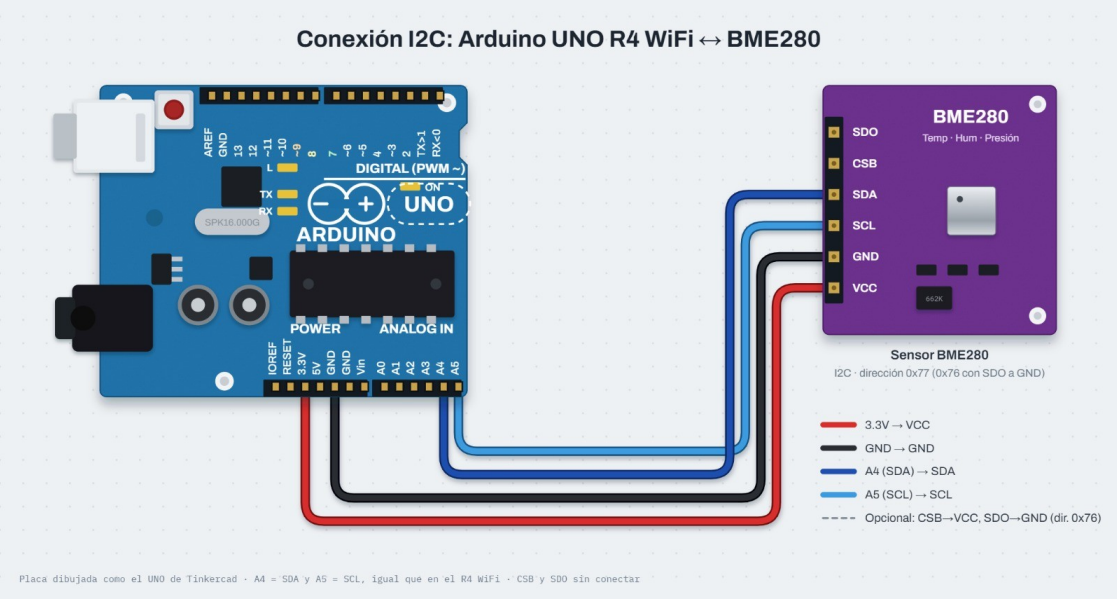


Figura 1. Diagrama de conexión I2C entre el Arduino y el sensor BME280/BMP280.

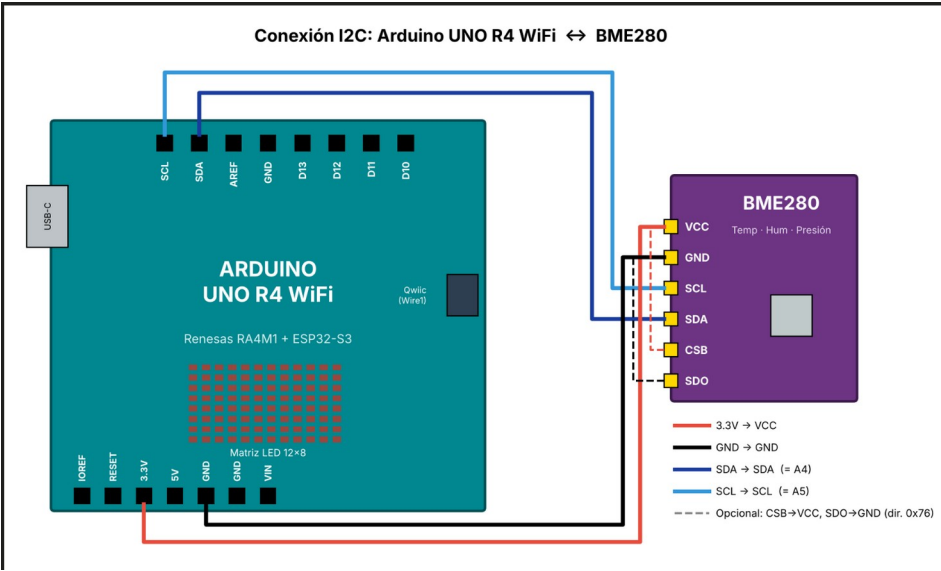


Figura 2. Conexión sobre el UNO R4 WiFi: SDA y SCL junto a AREF (equivalentes a A4 y A5) y alimentación de 3.3 V.

Tabla I. Conexiones del sensor con el Arduino UNO R4 WiFi.

Pin del sensor	Pin del Arduino	Función
VCC	3.3V	Alimentación del sensor.
GND	GND	Tierra común.
SCL	SCL (= A5)	Reloj del bus I2C, generado por el Arduino.
SDA	SDA (= A4)	Datos del bus I2C, en ambos sentidos.
CSB	—	Selección de modo. Sin conectar: el sensor trabaja en I2C.
SDO	—	Bit bajo de la dirección I2C (a GND = 0x76, a VCC = 0x77).

El programa (estacion_bmp280.ino) se organiza de la siguiente manera:

- **Detección del sensor:** `iniciarSensor()` prueba las direcciones 0x76 y 0x77. En cada una, `leerChipId()` escribe el número de registro 0xD0 y pide un byte con `requestFrom()`. Si el Chip ID es 0x58 (BMP280) se inicia la librería con `bmp.begin()`; si es 0x60 avisa que se trata de un BME280 y que debe usarse el sketch con la librería `Adafruit_BME280`.
- **Configuración del muestreo:** con `setSampling()` el sensor se pone en modo normal, con sobremuestreo $\times 2$ en temperatura y $\times 16$ en presión, filtro IIR $\times 16$ y una espera de 500 ms entre mediciones. Esto reduce el ruido de la presión, que es la que más influye en la altitud.
- **Lectura (cada 1000 ms):** `leerSensor()` obtiene la temperatura en $^{\circ}\text{C}$, la presión en Pa (que se divide entre 100 para pasarla a hPa) y la altitud con `readAltitude(SEALEVEL_HPA)`, usando una presión a nivel del mar de 1005.0 hPa. Los datos se imprimen como `Temp_C:..., Presion_Atm_hPa: ..., Altura_nivel_mar_m:...`, el formato etiqueta:valor que el Serial Plotter reconoce para graficar cada variable por separado.
- **Matriz LED (cada 2500 ms):** `mostrarMatriz()` escribe hasta tres caracteres con la fuente de 4×6 de `ArduinoGraphics` y alterna entre la temperatura redondeada seguida de “C” y la altitud seguida de “m”.
- **Validación y reconexión (cada 5000 ms):** si la temperatura o la presión salen como NaN, o la presión queda fuera de 300–1100 hPa, se muestra “Error: lectura invalida. Revisa el cableado.”, la matriz indica “ERR” y el programa intenta encontrar el sensor de nuevo cada 5 s, sin bloquearse.

Las tres tareas se controlan comparando `millis()` con el instante de la última ejecución de cada una:

```
if (ahora - tLectura >= T_LECTURA) {           // cada 1000 ms
    tLectura = ahora;
    leerSensor();
}
if (datosValidos && ahora - tMatriz >= T_MATRIZ) { // cada 2500 ms
    tMatriz = ahora;
    /* alterna temperatura (C) y altitud (m) en la matriz */
}
```

Al iniciar, el programa abre el puerto serie a 115200 baudios y espera hasta 3 s a que se abra el Monitor Serie con `while (!Serial && millis() - t0 < 3000)`, para no perder los primeros mensajes.

5. Evidencia del circuito funcionando

En las Figuras 3 y 4 se observa el sensor conectado al Arduino: los cables de SDA y SCL van a los pines junto a AREF y la alimentación a 3.3 V y GND. La matriz LED muestra un valor numérico y no el mensaje “ERR”, lo que indica que el programa encontró el sensor y está obteniendo lecturas válidas.

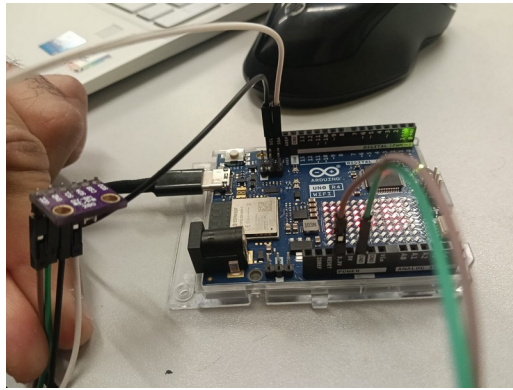


Figura 3. Sensor BMP280 conectado al bus I2C del Arduino UNO R4 WiFi, con la matriz LED mostrando un valor.

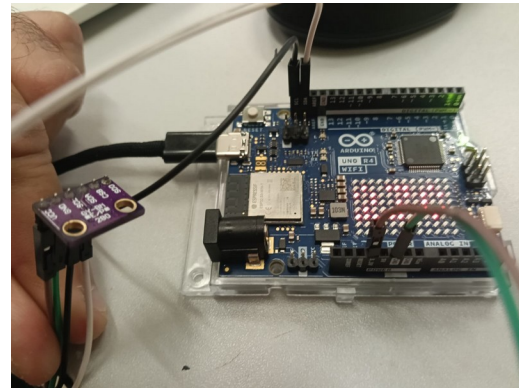


Figura 4. Otra vista del armado: conexiones a 3.3 V, GND, SDA y SCL.

En la Figura 5 se muestra el Monitor Serie durante una de las pruebas. En lugar de las lecturas aparecieron solo símbolos “❖”. Este problema se analiza en la sección 6.

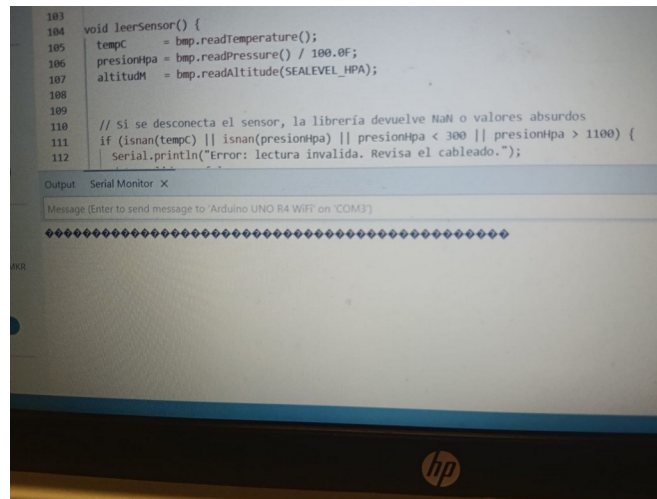


Figura 5. Monitor Serie con caracteres inválidos por una velocidad de comunicación distinta a la del programa.

El video con el funcionamiento de la práctica está disponible en: <https://youtu.be/ox5LbVQqwLA>

6. Análisis de resultados

Caracteres inválidos en el Monitor Serie

Los símbolos “❖” de la Figura 5 aparecen cuando el Monitor Serie trabaja a una velocidad distinta a la del programa. El sketch abre el puerto con `Serial.begin(115200)`, pero el Monitor estaba configurado en otra velocidad (por ejemplo, 9600 baudios, la que se usó en prácticas anteriores). En el UNO R4 WiFi el puerto USB-C está conectado al ESP32-S3, que funciona como puente USB-serie hacia el microcontrolador RA4M1, así que la velocidad sí tiene que coincidir. Con velocidades distintas, el receptor toma las muestras de cada bit en momentos equivocados, arma bytes que no corresponden a ningún carácter válido y el IDE los muestra como “❖”.

La solución es seleccionar **115200 baud** en la lista desplegable del Monitor Serie (o del Serial Plotter). El sensor no tenía ninguna falla: la matriz LED seguía mostrando valores, porque esa parte del programa no depende del puerto serie.

Efecto de la presión a nivel del mar en la altitud

El programa usa $P_0 = 1005.0$ hPa como presión a nivel del mar del día. La Tabla II muestra la altitud que calcula la fórmula de la librería para distintas presiones medidas con ese mismo valor de P_0 .

Tabla II. Altitud calculada con $h = 44330 \cdot [1 - (P / 1005.0)^{0.1903}]$.

Presión medida (hPa)	Altitud calculada (m)	Comentario
1005.00	0.0	Igual a P_0
1004.00	8.4	1 hPa menos
1003.00	16.8	2 hPa menos
1000.00	42.1	5 hPa menos
1013.25	-69.0	Presión estándar

Observaciones sobre el comportamiento del sistema:

- Cerca del nivel del mar, cada hPa de diferencia equivale a unos 8.4 m de altitud. Por eso, si P_0 no corresponde a la presión real del día, la altitud se desvía decenas de metros aunque el sensor mida bien; por ejemplo, con la presión estándar de 1013.25 hPa el programa reportaría -69 m.
- Mazatlán está prácticamente al nivel del mar, así que con un P_0 correcto la altitud debe salir cercana a 0 m. Para mejorarla, P_0 se puede actualizar con la presión a nivel del mar que publique una estación meteorológica ese día.
- La identificación por Chip ID permite saber qué sensor está conectado: como la matriz mostró valores y no “ERR”, el programa encontró un BMP280 (0x58); con un BME280 (0x60) se habría mostrado el aviso para cambiar de librería.
- El filtro IIR $\times 16$ y el sobremuestreo de presión $\times 16$ hacen que la lectura cambie de forma gradual, lo que evita que la altitud mostrada brinque varios metros entre una lectura y la siguiente.
- Gracias a `millis()`, las tareas de lectura, matriz y reconexión se ejecutan cada una a su propio ritmo sin bloquearse. Si el sensor se desconecta, el programa muestra “ERR” y lo vuelve a buscar cada 5 s en lugar de congelarse.
- El encabezado del documento de la práctica menciona la Adafruit BME280 Library, pero el sketch incluye `Adafruit_BMP280.h`; para compilarlo se necesita la **Adafruit BMP280 Library**.

7. Conclusiones individuales

Ximena

En esta práctica volví a usar el bus I2C, pero ahora con un sensor comercial en lugar de otro Arduino. Me pareció muy útil leer el Chip ID en el registro 0xD0, porque así el programa sabe si está conectado un BMP280 o un BME280 antes de iniciar la librería, y además busca solo en las dos direcciones posibles, 0x76 y 0x77.

También entendí que el BMP280 no mide la altitud directamente: la calcula a partir de la presión. Por eso es importante ajustar la presión a nivel del mar del día; con un valor equivocado la altitud puede desviarse decenas de metros aunque el sensor funcione bien.

Carlos Tadeo

El desarrollo de esta práctica me ayudó a entender cómo un solo programa puede atender varias tareas con `millis()`: leer el sensor cada segundo, cambiar la matriz LED cada 2.5 segundos y buscar el sensor cada 5 segundos si se desconecta, sin que ninguna tarea detenga a las demás.

Además, aprendí la importancia de la velocidad del puerto serie. Cuando el Monitor Serie no estaba en 115200 baudios solo aparecían símbolos raros, aunque el sensor y la matriz funcionaban bien. Ahora reviso que la velocidad del Monitor coincida con la de `Serial.begin()` antes de pensar que el circuito está mal.

Dulce Princesa

Considero que lo más interesante de esta práctica fue mostrar los datos de varias formas: en el Monitor Serie, en el Serial Plotter gracias al formato etiqueta:valor y en la matriz LED integrada del UNO R4 WiFi, que permite ver la temperatura y la altitud sin tener la computadora abierta.

Adicionalmente, me pareció importante que el programa valide cada lectura. Si el sensor se desconecta, la librería devuelve valores sin sentido, y gracias a la revisión del rango de 300 a 1100 hPa el sistema avisa con “ERR” y vuelve a buscar el sensor por sí solo, en lugar de mostrar datos falsos.

Referencias

[1] Bosch Sensortec, “BMP280 Digital Pressure Sensor”, hoja de datos. [En línea]. Disponible en: <https://www.bosch-sensortec.com/products/environmental-sensors/pressure-sensors/bmp280/>

[2] Bosch Sensortec, “BME280 Combined humidity and pressure sensor”, hoja de datos. [En línea]. Disponible en: <https://www.bosch-sensortec.com/products/environmental-sensors/humidity-sensors-bme280/>

[3] Adafruit, “Adafruit BMP280 Library”. [En línea]. Disponible en: https://github.com/adafruit/Adafruit_BMP280_Library

[4] Arduino, “Arduino UNO R4 WiFi”, documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc/hardware/uno-r4-wifi/>

[5] Arduino, “Using the Arduino UNO R4 WiFi LED Matrix”. [En línea]. Disponible en: <https://docs.arduino.cc/tutorials/uno-r4-wifi/led-matrix/>